Principal Component Analysis (PCA)

Prerequisite Theory, Mathematical Foundations, and Lab Walkthrough

AIML Module 2 • Lab 2

Pre-Lab Teaching Slides

Learning objectives

1. Explain why dimensionality reduction is useful.
2. Review vectors, matrices, projections, mean, variance and covariance.
3. Derive PCA using eigenvectors of a covariance matrix.
4. Understand explained variance and how to choose the number of components.
5. Implement PCA manually and with scikit-learn.
6. Interpret PCA visualizations of the breast cancer dataset.

Where this lab fits

Module 2: Appreciating, Interpreting and Visualizing Data

PCA is an unsupervised, linear transformation of numerical features to new orthogonal axes capturing maximal variance.

- Visualization: reduce many dimensions to 2D/3D.
- Feature reduction: compact representation for downstream ML.
- PCA does **not** use class labels to choose the axes.

Prerequisite 1: Data as a matrix

$$\mathbf{X} \in \mathbb{R}^{n \times d}, \quad \mathbf{X} = \begin{bmatrix} x_{11} & \cdots & x_{1d} \\ \vdots & & \vdots \\ x_{n1} & \cdots & x_{nd} \end{bmatrix}.$$

- n rows = observations; d columns = features.
- A row $\mathbf{x}_i \in \mathbb{R}^d$ is one sample.
- A column collects one measurement over all samples.
- In the tutorial: $n = 569$, $d = 30$; targets have shape $(569,)$.

Prerequisite 2: Vectors, dot products and lengths

$$\mathbf{a}^T\mathbf{b} = \sum_{j=1}^{d} a_j b_j, \qquad \|\mathbf{a}\|_2 = \sqrt{\mathbf{a}^T\mathbf{a}}.$$

- Dot product measures alignment; zero indicates orthogonality.
- Unit vectors satisfy $\|\mathbf{u}\|_2 = 1$.
- Scalar projection onto unit direction $\mathbf{u}$: $z = \mathbf{x}^T\mathbf{u}$.
- PCA outputs *coordinates along new directions*.

Quick check

For $\mathbf{x} = (3, 4)^T$ and $\mathbf{u} = (1, 0)^T$, what is $\mathbf{x}^T\mathbf{u}$? **Answer: 3**

Prerequisite 3: Mean, variance, standard deviation

$$\bar{x}_j = \frac{1}{n}\sum_{i=1}^{n} x_{ij}, \qquad s_j^2 = \frac{1}{n-1}\sum_i (x_{ij} - \bar{x}_j)^2.$$

$$s_j = \sqrt{s_j^2}, \qquad z_{ij} = \frac{x_{ij} - \bar{x}_j}{s_j}.$$

- Centering subtracts the feature mean.
- Scaling by standard deviation puts features on comparable units.
- `StandardScaler` uses a population-variance convention ($1/n$), not $1/(n-1)$.
- Standardization does **not** make an arbitrary feature normally distributed.

Why scale before PCA?

- Features such as radius, perimeter, area and texture have very different magnitudes.
- Without scaling, large-variance units can dominate principal directions.
- Scaling to zero mean/unit variance makes PCA effectively operate on correlations.
- Scaling is often helpful, *not an absolute requirement*: depends on whether original units carry meaningful variance.

Important pipeline rule

Fit the scaler on **training data only**; then transform validation/test data with the same scaler.

Prerequisite 4: Covariance

$$\mathrm{Cov}(x_j, x_k) = \frac{1}{n-1} \sum_{i=1}^{n} (x_{ij} - \bar{x}_j)(x_{ik} - \bar{x}_k).$$

- Positive: tend to increase together.
- Negative: tend to move in opposite directions.
- Near zero: no strong *linear* co-variation.
- Covariance depends on units; correlation rescales it to $[-1, 1]$.

The covariance matrix

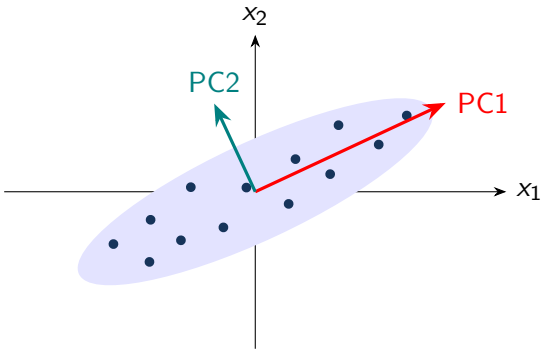
$$\mathbf{C} = \frac{1}{n-1}\mathbf{X}_c^T\mathbf{X}_c \in \mathbb{R}^{d\times d}, \quad \mathbf{X}_c = \mathbf{X} - \mathbf{1}\bar{\mathbf{x}}^T.$$

- Diagonal: variances of individual features.
- Off diagonal: pairwise covariance.
- $\mathbf{C}$ is symmetric and positive semidefinite.
- Breast cancer example: $\mathbf{C}$ has shape 30×30.

NumPy sanity check

`np.cov(X_std.T)` should match the manually computed covariance matrix.

Geometric intuition: find the longest spread



PCA rotates axes to the direction of largest variance, then the largest remaining orthogonal variance.

Prerequisite 5: Eigenvectors and eigenvalues

$$\mathbf{C}\mathbf{v}_j = \lambda_j \mathbf{v}_j, \quad \|\mathbf{v}_j\|_2 = 1.$$

- Eigenvector $\mathbf{v}_j$: direction unchanged by transformation $\mathbf{C}$ (up to scale).
- Eigenvalue λ_j: variance along the corresponding PCA axis.
- Symmetric covariance matrices have orthonormal eigenvectors.
- Sort eigenvalues $\lambda_1 \geq \cdots \geq \lambda_d \geq 0$.

PCA optimization: first principal component

Goal

Find a unit direction $\mathbf{w}$ maximizing the variance of projected centered data.

$$\max_{\|\mathbf{w}\|_2=1} \mathrm{Var}(\mathbf{X}_c\mathbf{w}) = \max_{\mathbf{w}^T\mathbf{w}=1} \mathbf{w}^T\mathbf{C}\mathbf{w}.$$

Lagrangian:

$$\mathcal{L}(\mathbf{w}, \lambda) = \mathbf{w}^T\mathbf{C}\mathbf{w} - \lambda(\mathbf{w}^T\mathbf{w} - 1).$$

$$\nabla_{\mathbf{w}}\mathcal{L} = 2\mathbf{C}\mathbf{w} - 2\lambda\mathbf{w} = 0 \;\Rightarrow\; \boxed{\mathbf{C}\mathbf{w} = \lambda\mathbf{w}}.$$

Choose the eigenvector with the **largest eigenvalue**.

Subsequent PCs and orthogonality

- PC2 maximizes remaining projected variance subject to $\mathbf{w}_2^T \mathbf{w}_1 = 0$.
- Continuing yields mutually perpendicular axes $\mathbf{w}_1, \ldots, \mathbf{w}_k$.
- Loadings/projection matrix: $\mathbf{W}_k = [\mathbf{w}_1 \cdots \mathbf{w}_k] \in \mathbb{R}^{d \times k}$.
- Transformed scores: $\boxed{\mathbf{Z} = \mathbf{X}_c \mathbf{W}_k \in \mathbb{R}^{n \times k}}$.

Shape check

For 569 samples, 30 input features and 3 PCs: $(569 \times 30)(30 \times 3) = (569 \times 3)$.

Worked numerical example

$$\mathbf{C} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix}.$$

$$\lambda_1 = 5, \quad \mathbf{v}_1 = \tfrac{1}{\sqrt{2}}(1,1)^T; \qquad \lambda_2 = 1, \quad \mathbf{v}_2 = \tfrac{1}{\sqrt{2}}(1,-1)^T.$$

- PC1 along the diagonal $x_1 = x_2$, capturing $5/(5+1) = 83.3\%$ of variance.
- PC2 captures 16.7%.
- One-component PCA projects $\mathbf{x} = (2,0)^T$ to score $\mathbf{v}_1^T\mathbf{x} = \sqrt{2}$.

Explained variance and cumulative variance

$$r_j = \frac{\lambda_j}{\sum_{\ell=1}^{d} \lambda_\ell}, \qquad R_k = \sum_{j=1}^{k} r_j.$$

- r_j: fraction explained by PC j.
- R_k: fraction retained by first k PCs.
- Plot r_j (scree plot) and R_k to choose k.
- Tutorial reports roughly 44% for PC1, $\sim 63\%$ for first two, $\sim 73\%$ for first three after scaling.

Do not overclaim

Variance retained is *not* equal to classification accuracy or necessarily "signal" retained.

How many components should we retain?

1. **Visualization:** choose $k = 2$ or 3 for plotting.
2. **Variance target:** smallest k with $R_k \geq 0.90$ (or a task-specific threshold).
3. **Scree elbow:** balance simplicity with diminishing variance gain.
4. **Prediction:** tune k using cross-validation and downstream metrics.

Lab exercise

Find the smallest number of PCs retaining at least 90% variance. Do not guess from the 2D scatter plot.

PCA and reconstruction error

$$\hat{\mathbf{X}}_c = \mathbf{Z}\mathbf{W}_k^T = \mathbf{X}_c\mathbf{W}_k\mathbf{W}_k^T.$$

$$\min_{\mathbf{W}_k^T\mathbf{W}_k=\mathbf{I}} \|\mathbf{X}_c - \mathbf{X}_c\mathbf{W}_k\mathbf{W}_k^T\|_F^2.$$

- Keeping the top k eigenvectors minimizes squared reconstruction error among rank-k orthogonal projections.
- Discarded covariance eigenvalues sum to the average squared reconstruction error.
- Add the original mean back to reconstruct in original centered units (and invert scaling if applied).

Connection to singular value decomposition (SVD)

$$\mathbf{X}_c = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T, \qquad \mathbf{C} = \mathbf{V}\frac{\mathbf{\Sigma}^2}{n-1}\mathbf{V}^T.$$

- Columns of $\mathbf{V}$ are principal directions.
- Eigenvalues $\lambda_j = \sigma_j^2/(n-1)$.
- SVD often avoids explicitly forming $\mathbf{C}$ and is numerically preferable.
- `sklearn.decomposition.PCA` uses SVD-based solvers in common settings.

Lab dataset: breast cancer measurements

- `load_breast_cancer()` from scikit-learn.
- **569 samples**, **30 numeric features** describing nuclei in breast mass images.
- Classes: **0 = malignant**, **1 = benign** (verify `target_names`).
- Labels are used to *color the visualization*, not to compute PCA.

Correction to the supplied notebook

Its descriptive text and class-label mapping reverse the actual scikit-learn target encoding. Fix plot labels before teaching.

Lab step 1: load, inspect, standardize

```
from sklearn.datasets import load_breast_cancer
from sklearn.preprocessing import StandardScaler

data = load_breast_cancer()
X, y = data.data, data.target
print(X.shape, y.shape)  # (569, 30), (569,)
print(data.target_names) # ['malignant', 'benign']
X_std = StandardScaler().fit_transform(X)
```

- Inspect feature names, distributions and missing values.
- For predictive evaluation use a train/test split *before* fitting preprocessing.

Lab step 2: covariance + eigen-decomposition

```
import numpy as np
C = np.cov(X_std, rowvar=False)
values, vectors = np.linalg.eigh(C)
order = np.argsort(values)[::-1]
values = values[order]
vectors = vectors[:, order]
ratios = values / values.sum()
cumulative = np.cumsum(ratios)
```

- `eigh` exploits the symmetry of covariance; appropriate here.
- Eigenvectors are *columns* of `vectors`.
- Do not rank eigenvalues by absolute magnitude; covariance eigenvalues should be nonnegative (up to numerical error).

Lab step 3: project to 3 principal components

```
W3 = vectors[:, :3]    # (30, 3)
Z_manual = X_std @ W3 # (569, 3)
print(Z_manual.shape)
print(cumulative[2])  # variance retained by 3 PCs
```

What changed?

30 original axes were replaced by 3 orthogonal combinations of the original features. The 569 observations remain unchanged in count.

Lab step 4: scikit-learn PCA

```
from sklearn.decomposition import PCA
pca = PCA(n_components=3)
Z_sklearn = pca.fit_transform(X_std)
print(pca.explained_variance_ratio_)
print(pca.explained_variance_ratio_.sum())
```

- Manual and sklearn principal *subspaces* should agree.
- Corresponding component signs may differ: $\mathbf{v}$ and $-\mathbf{v}$ are equivalent.
- Compare explained variance and projection correlations rather than raw signs.

Lab step 5: 2D and 3D visualization

```
import matplotlib.pyplot as plt
for label, name in enumerate(data.target_names):
    m = (y == label)
    plt.scatter(Z_sklearn[m,0], Z_sklearn[m,1],
                label=name, alpha=.6, s=16)
plt.xlabel('PC1'); plt.ylabel('PC2'); plt.legend()
plt.show()
```

- For 3D, plot PC1, PC2, PC3 with Plotly as in the tutorial.
- Color by ground-truth class *after* unsupervised PCA.
- Visible separation is descriptive, not proof of generalization.

Common conceptual mistakes

1. PCA is **not** supervised classification; labels do not inform directions.
2. PCA is not guaranteed to preserve class separation.
3. High explained variance does not automatically imply high model accuracy.
4. Standardization does not enforce a Gaussian distribution.
5. Eigenvector *sign flips* are expected and harmless.
6. PCA captures **linear** structure and can lose low-variance discriminative signals.

Practical machine-learning tips

- Use `Pipeline(StandardScaler(), PCA(), classifier)` to avoid data leakage.
- Choose number of PCs on training/validation folds only.
- Compare no-PCA vs PCA baselines using the same split and metric.
- Keep feature names and inspect component loadings for interpretability.
- Watch for outliers, since variance-based PCA is sensitive to them.
- Use scatter-plot transparency and report explained variance on axes.

Optional: leakage-safe predictive pipeline

```
from sklearn.pipeline import make_pipeline
from sklearn.model_selection import cross_val_score
from sklearn.linear_model import LogisticRegression

model = make_pipeline(StandardScaler(),
                      PCA(n_components=0.90),
                      LogisticRegression(max_iter=2000))
scores = cross_val_score(model, X, y, cv=5,
                         scoring='accuracy')
print(scores.mean(), scores.std())
```

`n_components=0.90` asks the default full-SVD PCA solver to retain at least 90% variance. Compare against a no-PCA pipeline.

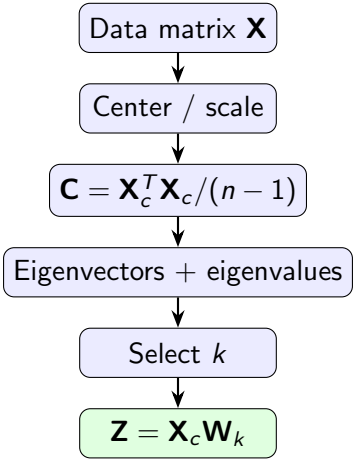
Think–pair–share questions

1. Why must PCA axes be orthogonal?
2. Why does scaling change the direction of PC1?
3. If PC1 explains 44% and PC2 19%, how much do both explain?
4. Why can two valid PCA implementations return opposite signs for the same axis?
5. If a 2D projection separates classes, is PCA a classifier?
6. How would you choose k for classification rather than plotting?

Student exercises

1. Compute the smallest k for at least **90% cumulative explained variance**.
2. Compare PCA **with vs without scaling**. What changes?
3. Verify `np.cov(X_std.T)` against the matrix formula.
4. Compare manual PCA with scikit-learn including sign flips.
5. Research one additional PCA use case, explaining the inputs, outputs and limitations.

Summary: the PCA pipeline



References and next steps

- Primary source: **AIML Module 2 – Lab 2 PCA** (provided notebook).
- Scikit-learn documentation: `sklearn.decomposition.PCA`, `StandardScaler`, `load_breast_cancer`.
- Jolliffe and Cadima (2016), *Principal component analysis: a review and recent developments*.

Lab-ready checklist

Students can explain standardization, covariance, eigenvectors, explained variance, projection, and how to interpret the 2D/3D plots.